TDP — TRADUTOR DE PALAVRAS

Plano de Gerenciamento de Configuração

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.0

Projeto	TDP — Tradutor de Palavras
Gerente de Projeto	Felipe Silveira
Responsável de GCS-DevOps	Felipe Silveira
Fábrica de Software / Squad	Individual — Felipe Silveira
Repositório principal	`https://github.com/fleurissent/tradutor`
Disciplina	Gerência de Configuração de Software

Histórico de Revisões

Data	Versão	Descrição	Autor
22/09/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD), aplicado ao app Tradutor de Palavras.	Felipe Silveira

Sumário

1. Introdução
2. Cultura DevOps e Papéis na Gerência de Configuração
3. Ferramentas, Ambientes e Infraestrutura
4. Identificação da Configuração
5. Integração Contínua (CI)
6. Entrega e Implantação Contínua (CD)
7. Infraestrutura como Código (IaC)
8. Controle de Configuração e Mudança
9. Baselines e Releases
10. Estimativa do Status de Configuração
11. Marcos
12. Treinamento e Recursos
13. Controle de Dependências de Terceiros
14. Quadro-Resumo do Pipeline CI/CD

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) executadas ao longo do ciclo de vida do aplicativo **Tradutor de Palavras**, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pelo desenvolvedor responsável pelo projeto, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, análise e empacotamento.

1.2 Escopo

Este PGC é destinado ao desenvolvimento individual, por Felipe Silveira, do aplicativo **Tradutor de Palavras**: um app mobile Flutter/Dart que exibe um vocabulário em Português e, sob demanda, suas traduções em Inglês e Espanhol, com leitura da palavra em voz alta (text-to-speech). O plano abrange o código-fonte, os testes, as configurações de build Android e os pipelines de automação do projeto.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-Based Development	Estratégia de branching em que se integra na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) — ex.: 1.0.0

TTS	Text-to-Speech (síntese de voz), provido pela biblioteca `flutter_tts`
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado com enfoque DevOps.
- Código-fonte do projeto: `https://github.com/fleurissent/tradutor`
- Documentação da biblioteca `flutter_tts` (pub.dev).

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: cada commit integrado, testado e empacotado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados — o APK), reduzindo o esforço manual e o tempo entre a mudança e sua disponibilização.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto DevOps é responsabilidade compartilhada entre desenvolvimento e operações ("você constrói, você executa"), apoiada por automação. Por se tratar de um projeto individual, **todos os papéis abaixo são exercidos por Felipe Silveira**; a descrição por papel é mantida para deixar explícitas as responsabilidades cobertas.

Papel	Responsabilidades	Responsável
Gerente de Projeto / Product Owner	Prioriza o backlog, autoriza mudanças de maior impacto e acompanha indicadores de entrega.	Felipe Silveira
Engenheiro DevOps / Release Engineer	Projeta e mantém o pipeline de CI, define a estratégia de branching e versionamento e a geração de artefatos (APK).	Felipe Silveira
Desenvolvedor(a)	Integra código com frequência na branch principal, escreve testes automatizados e é corresponsável pela qualidade.	Felipe Silveira
QA / Automação de Testes	Mantém a suíte de testes de widget executada no pipeline de CI.	Felipe Silveira
Segurança (DevSecOps)	Acompanha a análise de dependências (SCA) e mantém as bibliotecas de terceiros atualizadas.	Felipe Silveira
Comitê de Controle de Mudanças (CCM)	Aciona-se apenas em mudanças de alto risco/impacto; mudanças de rotina seguem o fluxo automatizado de PR + pipeline.	Felipe Silveira (autoaprovação documentada)

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas efetivamente adotadas neste projeto:

Categoria	Ferramenta adotada	Finalidade
Controle de versão	Git + GitHub	Versionamento do código-fonte, testes e definições de pipeline
Orquestração de CI/CD	GitHub Actions	Automação de build, análise, testes e empacotamento
Build / SDK	Flutter SDK (Dart) + Gradle (Android)	Compilação da aplicação e geração do APK
Empacotamento	APK Android (Gradle)	Artefato versionado e instalável do aplicativo
Registro de artefatos	GitHub Releases / Artefatos de workflow	Armazenamento e versionamento do APK gerado
Análise estática	`flutter analyze` + `flutter_lints`	Verificação de padrões de codificação (lint)
Testes automatizados	`flutter_test`	Testes de widget executados no pipeline
Gestão de dependências / SCA	Dependabot (GitHub)	Identificação de vulnerabilidades e atualização de bibliotecas
Rastreamento de tarefas	GitHub Issues	Vinculação de commits/PRs a itens de trabalho

> **Observação:** por ser um aplicativo mobile sem backend próprio, não há uso de contêineres (Docker/Kubernetes), IaC de servidores (Terraform/Ansible) nem cofre de segredos de infraestrutura. Esses itens do template são **Não Aplicáveis (N/A)** e estão tratados como tal na seção 7.

3.2 Ambientes e Infraestrutura

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento (local / emulador)	Integração contínua do código do desenvolvedor; execução via `flutter run`.	Commit em branch de feature
Integração (GitHub Actions)	Build, análise e testes automatizados; geração do APK de validação.	Abertura/atualização de PR e merge na `main`

Distribuição (GitHub Releases)	Disponibilização do APK estável aos usuários/avaliadores.	Criação de tag `vX.Y.Z` após validação

3.3 Estratégia de Branching e Versionamento

Adota-se **Trunk-Based Development** (GitHub Flow), com branches de vida curta:

- `main` — sempre implantável; protegida por regra de PR obrigatório e pipeline verde.
- `feature/<descrição-curta>` — branches de curta duração (idealmente < 1–2 dias) por mudança.
- `hotfix/<descrição>` — correções emergenciais, retro-integradas ao trunk.

Versionamento: Versionamento Semântico (MAJOR.MINOR.PATCH — atualmente `1.0.0`) e **Conventional Commits** (`feat`, `fix`, `docs`, `test`, `chore`, `ci`...), permitindo geração automática de changelog e da próxima versão.

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, os seguintes artefatos são itens de configuração formais, versionados no repositório:

Item de configuração	Arquivo(s) / Local
Código-fonte da aplicação	`lib/main.dart`
Testes automatizados	`test/widget_test.dart`
Definições de pipeline (pipeline como código)	`.github/workflows/ci.yml`
Configuração de dependências e versão	`pubspec.yaml`
Dependências travadas (lockfile)	`pubspec.lock`
Regras de análise estática	`analysis_options.yaml`
Configuração de build Android	`android/ (Gradle, AndroidManifest.xml)`
Documentação técnica	`README.md, CHANGELOG.md, docs/adr/`
Scripts de migração de banco de dados	N/A — o app não possui persistência/backend

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	`feature/<descrição-curta>`	`feature/adicionar-frances`
Commit	`<tipo>(<escopo>): <descrição>`	`feat(tradutor): adiciona idioma francês`
Tag de release	`vX.Y.Z`	`v1.0.0`
Artefato (APK)	`tradutor-<versão>.apk`	`tradutor-1.0.0.apk`

4.3 Estrutura do Repositório

Diretório / Arquivo	Conteúdo
`/lib`	Código-fonte da aplicação (`main.dart`)
`/test`	Testes automatizados de widget
`/android`	Projeto Android (manifests, configuração Gradle)

`/.github/workflows`	Pipelines de CI (build, lint, testes, APK)
`/docs/adr`	Architecture Decision Records
`pubspec.yaml / pubspec.lock`	Dependências e versão do projeto

5. Integração Contínua (CI)

Todo push e todo Pull Request para a branch principal dispara automaticamente o pipeline de CI (definido em `.github/workflows/ci.yml`). Nenhuma mudança é integrada sem que o pipeline seja concluído com sucesso.

5.1 Etapas do Pipeline de CI

1. Checkout do código e configuração do Flutter (com cache de dependências).
2. Restauração de dependências (`flutter pub get`).
3. Verificação de formatação (`dart format --set-exit-if-changed`).
4. Análise estática / lint (`flutter analyze`).
5. Testes de widget com medição de cobertura (`flutter test --coverage`).
6. Build do artefato (`flutter build apk --release`).
7. Publicação do APK e do relatório de cobertura como artefatos do workflow.

5.2 Política de Pull Request e Revisão de Código

- Todo PR exige, no mínimo, 1 aprovação antes do merge (revisão "quatro olhos"). Em projeto individual, adota-se autorrevisão criteriosa registrada no PR e/ou revisão por colega quando disponível.
- PRs devem ser pequenos e de vida curta, para reduzir conflitos e facilitar a revisão.
- Merge somente permitido com pipeline de CI verde (quality gate) — configurado via branch protection na `main`.

5.3 Critérios de Qualidade (Quality Gate)

Critério	Limite objetivo
Erros de análise estática (lint)	Zero erros bloqueantes
Testes automatizados	100% dos testes passando
Formatação de código	`dart format` sem diferenças
Build do APK	Concluído com sucesso
Cobertura de testes	Meta ≥ 60% das linhas do código de aplicação

6. Entrega e Implantação Contínua (CD)

Após a validação pelo pipeline de CI, a entrega do aplicativo é feita pela publicação do APK versionado. Por se tratar de um app mobile (e não de um serviço em execução contínua), a "implantação" corresponde à **distribuição do APK** ao usuário/avaliador, e não a um deploy em servidores.

6.1 Etapas da Entrega

1. Merge aprovado na `main` com pipeline verde.
2. Criação de uma tag de versão (`vX.Y.Z`) — baseline do produto.
3. Build do APK de release a partir daquela versão exata (sem recompilação divergente).
4. Publicação do APK em GitHub Releases, com notas de versão (changelog).
5. Verificação de instalação/execução (smoke test manual em dispositivo/emulador).

6.2 Estratégias de Implantação (aplicabilidade ao contexto mobile)

Estratégia	Descrição	Aplicabilidade
Rolling update	Substituição gradual de instâncias.	N/A — não há servidores
Blue-Green	Duas versões coexistem; tráfego comutado de uma vez.	N/A
Canary Release	Liberação gradual a uma fração dos usuários.	Aplicável em produção real via faixas da Play Store (staged rollout); fora do escopo acadêmico atual
Feature Flags	Funcionalidades implantadas desligadas e ativadas depois.	Aplicável a evoluções futuras; não usado na v1.0.0

6.3 Rollback e Recuperação

Toda entrega é reversível: como cada release é uma tag e cada APK é preservado em GitHub Releases, o rollback consiste em redistribuir o APK da versão anterior estável. O código da versão anterior permanece sempre recuperável a partir da tag correspondente.

7. Infraestrutura como Código (IaC)

Este projeto é um aplicativo mobile **sem infraestrutura de servidores própria** (sem backend, banco de dados ou serviços em nuvem). Portanto, provisionamento de servidores, Terraform/Ansible, ambientes efêmeros e state files são **Não Aplicáveis (N/A)**.

O equivalente a "infraestrutura como código" neste contexto é a **configuração de build e do pipeline versionada no repositório**: os arquivos do diretório `android/` (Gradle, `AndroidManifest.xml`) e o workflow `.github/workflows/ci.yml` descrevem, como código versionado, como o ambiente de build é montado e executado — submetidos ao mesmo fluxo de Pull Request e pipeline da aplicação.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina (funcionalidades, correções) seguem o fluxo padrão: abertura de um item de trabalho (GitHub Issue) → branch de feature → Pull Request → revisão → pipeline de CI → merge → tag/release. Esse fluxo, por si, já constitui o registro formal e rastreável da mudança.

8.2 Comitê de Controle de Mudança (CCM)

Por ser um projeto individual, o CCM formal é substituído pela revisão criteriosa do próprio autor no PR. Ele seria acionado apenas em mudanças de alto impacto (ex.: alteração estrutural do app, mudança de dependência crítica ou de plataforma-alvo). Mudanças rotineiras seguem o fluxo automatizado descrito em 8.1.

9. Baselines e Releases

9.1 Geração de Baseline

Cada versão estável marcada com tag SemVer constitui automaticamente uma baseline, associando o commit, o APK publicado e as notas de versão correspondentes. A primeira baseline do produto é a **v1.0.0**.

9.2 Notas de Versão (Release Notes)

As notas de release são derivadas dos Conventional Commits desde a última tag e registradas no `CHANGELOG.md` e na respectiva GitHub Release.

10. Estimativa do Status de Configuração

10.1 Monitoramento

Como não há ambiente de produção em servidores, o "status de configuração" é monitorado pelo histórico de execuções do pipeline (builds, testes) e pelo estado das branches e tags no GitHub, que evidenciam, a qualquer momento, qual versão está estável e o que está em desenvolvimento.

10.2 Métricas DORA (adaptadas)

Métrica	Descrição	Como é obtida neste projeto
Deployment Frequency	Frequência de novas releases.	Nº de tags/releases por período
Lead Time for Changes	Tempo entre o commit e a release.	Data do commit × data da tag
Change Failure Rate	% de releases que falham.	Nº de hotfixes ÷ nº de releases
Mean Time to Restore (MTTR)	Tempo médio para corrigir uma falha.	Tempo entre detecção e release de correção

10.3 Relatórios e Auditorias

O histórico de pipelines (builds, testes) e os logs do Git (commits, PRs, tags) constituem, por si, o principal insumo de auditoria de GCS, dispensando relatórios manuais.

11. Marcos

Marco	Descrição	Status
M1 — Repositório versionado	Projeto sob controle de versão Git com histórico em Conventional Commits.	Concluído
M2 — Testes automatizados	Suíte de testes de widget do fluxo principal.	Concluído
M3 — Pipeline de CI	Build, lint, testes e geração de APK automatizados.	Concluído
M4 — Primeira baseline (v1.0.0)	Tag e release da primeira versão estável.	Concluído
M5 — Branch protection + Dependabot	Proteção da `main` e análise de dependências.	Planejado

12. Treinamento e Recursos

As ferramentas necessárias (Flutter SDK, Git, GitHub, GitHub Actions) são gratuitas e já dominadas pelo desenvolvedor. Os recursos de aprendizagem incluem a documentação oficial do Flutter, do GitHub Actions e do pub.dev. Não há necessidade de treinamento adicional de equipe, por se tratar de projeto individual.

13. Controle de Dependências de Terceiros

Bibliotecas de terceiros também são itens de configuração e são versionadas explicitamente:

Dependência	Versão	Finalidade
`flutter_tts`	^4.2.5	Leitura da palavra em voz alta (text-to-speech)
`cupertino_icons`	^1.0.8	Ícones no estilo iOS
`flutter_lints`	^6.0.0 (dev)	Conjunto de regras de análise estática

- As versões são travadas no `pubspec.lock`, garantindo builds reprodutíveis.
- Análise de vulnerabilidades e propostas de atualização são feitas automaticamente pelo **Dependabot**, submetidas ao mesmo fluxo de PR + CI.

14. Quadro-Resumo do Pipeline CI/CD

Etapa	Descrição	Gatilho
CI	Build, formatação, lint, testes de widget e geração do APK.	Abertura/atualização de Pull Request
Merge	Integração da mudança na branch principal.	Aprovação de PR + CI verde
Baseline / Tag	Criação de tag SemVer (`vX.Y.Z`).	Decisão de release
Distribuição	Publicação do APK em GitHub Releases + notas de versão.	Criação da tag
Verificação	Smoke test de instalação/execução do APK.	Após a distribuição

> Este plano mantém os quatro pilares da GCS — identificação, controle de versão, controle de mudanças e auditoria/status — agora suportados por automação, adaptados ao contexto de um aplicativo mobile Flutter desenvolvido individualmente.